

Razorpay Integration & Downstream Rail Contracts

Verified Sandbox Integration & Idempotency Controls

1. Razorpay Rail Interfaces

Interface	Method / Path	ACG Security & Control Mechanism	Verified Status
Order Creation	POST /v1/orders	Idempotency bound via receipt = intent_id	VERIFIED (Sandbox & Mock)
Webhook Ingress	POST /webhooks/razorpay	Constant-time HMAC SHA-256 + Event ID deduplication	VERIFIED (Live HTTP)
Idempotent Refund	POST /v1/payments/{id}/refund	X-Refund-Idempotency header; pre-capture lockout	VERIFIED (Live HTTP)
Payment Status Sync	GET /v1/payments/{id}	Outbox state reconciliation	VERIFIED

2. Idempotency Contract (receipt = intent_id)

When an agent initiates a checkout, ACG assigns the client's `intent_id` as the Razorpay order `receipt`.

- If the client retries an identical `intent_id`, ACG intercepts the duplicate at the session gate (HTTP 409 `DUPLICATE_INTENT_REPLAY`).
- If a transient network failure occurs during transmission to Razorpay, Razorpay matches on `receipt` and returns the existing order object rather than minting a duplicate.

3. Webhook Monotonic State Transition

Incoming Razorpay webhooks drive monotonic state updates in the `order_sessions` table:

- * `payment.captured` \rightrightarrows State advances to `PAYMENT_CAPTURED`; reservation is permanently committed; fulfillment is dispatched.
- * `payment.failed` \rightrightarrows State advances to `PAYMENT_FAILED`; held budget and inventory are rolled back immediately.
- * **Deduplication:** The `processed_webhook_events` table ensures that repeated delivery of the same `x-razorpay-event-id` returns HTTP 200 `DUPLICATE_IGNORED` without mutating state twice.

4. Refund Security & Capture Requisite

ACG enforces a strict capture requisite on refunds. If an order has not reached `PAYMENT_CAPTURED` or lacks a valid `razorpay_payment_id`, refund requests are strictly rejected, preventing illegitimate financial leakage.